MIT Academy of Engineering, Alandi(D), Pune
Department of CSE AIML

Assignment – 6

Implement Queue using linked list data Structure and Implement Queue using two Stacks

Student Name: Vivek Vaidya

PRN: 202501110081

Batch: B1

Advanced Data Structures: Queue Implementations

Part 1: Implementing a Queue Using a Linked List

1. Conceptual Overview

• Queue Definition: A First-In-First-Out (FIFO) linear data structure where elements are added at the back (enqueue) and removed from the front (dequeue)

• Linked List Approach: Uses dynamic memory nodes with front and rear pointers to achieve efficient O(1) insertions and deletions without fixed-size array constraints

2. C++ Implementation

```
#include <iostream>

#include <stdexcept>


struct Node {

   int data;

   Node* next;

   Node(int val) : data(val), next(nullptr) {}

};


class QueueUsingLinkedList {

private:

   Node* frontNode;
```

```
    Node* rearNode;

public:
    QueueUsingLinkedList() : frontNode(nullptr), rearNode(nullptr) {}

    void enqueue(int x) {
        Node* newNode = new Node(x);
        if (rearNode == nullptr) {
            frontNode = rearNode = newNode;
            return;
        }
        rearNode->next = newNode;
        rearNode = newNode;
    }

    int dequeue() {
        if (frontNode == nullptr) {
            throw std::out_of_range("Queue Underflow");
        }
        Node* temp = frontNode;
        int val = temp->data;
        frontNode = frontNode->next;
        if (frontNode == nullptr) {
            rearNode = nullptr;
        }
        delete temp;
```

```
        return val;
    }

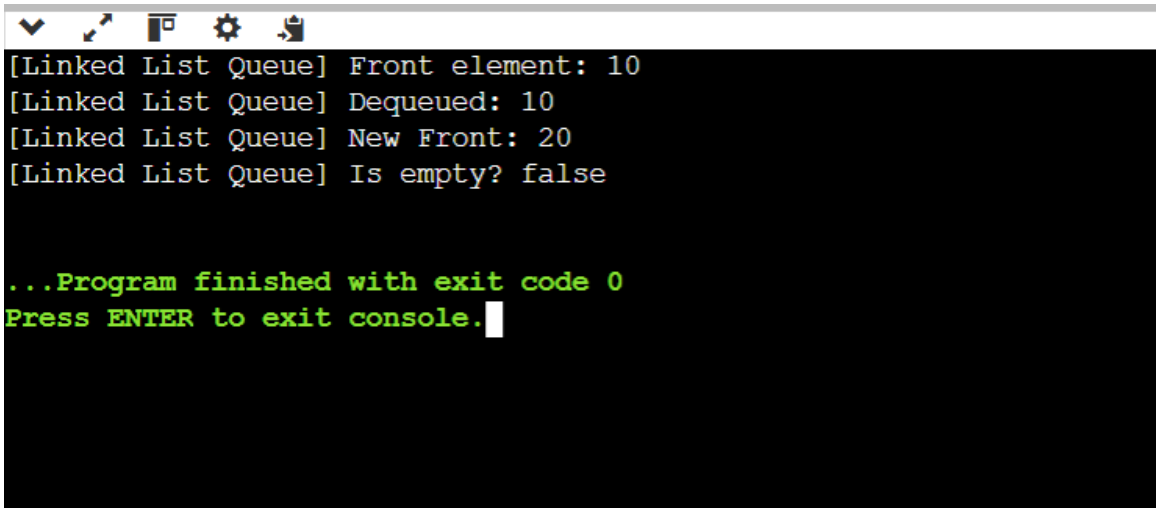
    int peek() {
        if (frontNode == nullptr) {
            throw std::out_of_range("Queue is empty");
        }
        return frontNode->data;
    }

    bool empty() {
        return frontNode == nullptr;
    }
};

int main() {
    QueueUsingLinkedList q;
    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);

    std::cout << "[Linked List Queue] Front element: " << q.peek() << std::endl;
    std::cout << "[Linked List Queue] Dequeued: " << q.dequeue() << std::endl;
    std::cout << "[Linked List Queue] New Front: " << q.peek() << std::endl;
    std::cout << "[Linked List Queue] Is empty? " << (q.empty() ? "true" : "false") << std::endl;
```

```
    return 0;
}
```


[Linked List Queue] Front element: 10
[Linked List Queue] Dequeued: 10
[Linked List Queue] New Front: 20
[Linked List Queue] Is empty? false
...Program finished with exit code 0
Press ENTER to exit console.

Part 2: Implementing a Queue Using Two Stacks

1. Conceptual Overview

• Two-Stack Approach: Leverages the LIFO property of two stacks (s1 and s2); pushing elements into s1 and transferring them to s2 reverses their order, perfectly converting LIFO behavior into FIFO behavior.

• Stack Layout: Uses s1 for incoming elements (enqueue) and s2 for outgoing elements (dequeue/peek)

2. C++ Implementation

```
#include <stack>
#include <iostream>
#include <stdexcept>

class QueueUsingTwoStacks {
private:
    std::stack<int> s1;
    std::stack<int> s2;

    void shiftStack() {
        while (!s1.empty()) {
            s2.push(s1.top());
            s1.pop();
        }
    }

public:
    void enqueue(int x) {
        s1.push(x);
    }

    int dequeue() {
        if (s2.empty()) {
            if (s1.empty()) {
                throw std::out_of_range("Queue Underflow");
```

```
        }
        shiftStack();
    }
    int val = s2.top();
    s2.pop();
    return val;
  }

  int peek() {
    if (s2.empty()) {
      if (s1.empty()) {
        throw std::out_of_range("Queue is empty");
      }
      shiftStack();
    }
    return s2.top();
  }

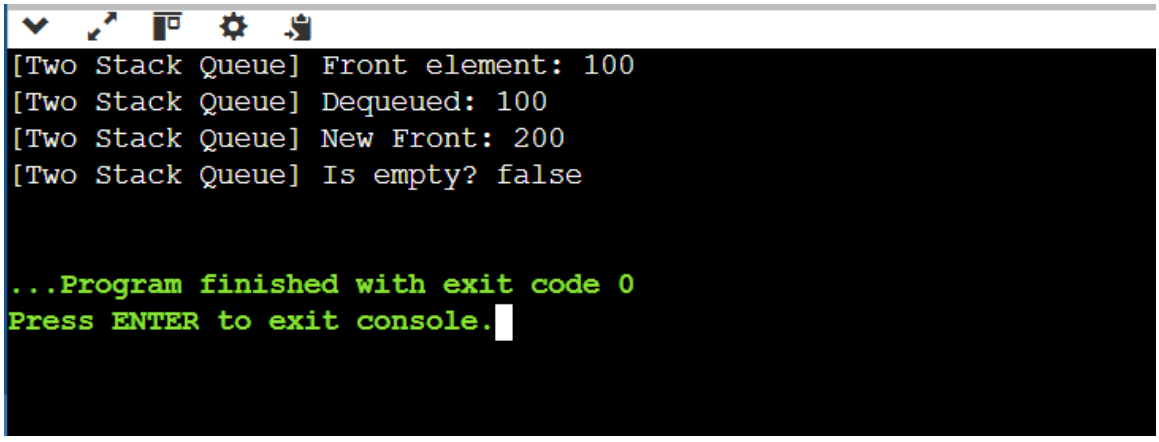
  bool empty() {
    return s1.empty() && s2.empty();
  }
};

int main() {
  QueueUsingTwoStacks q;
  q.enqueue(100);
```

```
    q.enqueue(200);
    q.enqueue(300);

    std::cout << "[Two Stack Queue] Front element: " << q.peek() << std::endl;
    std::cout << "[Two Stack Queue] Dequeued: " << q.dequeue() << std::endl;
    std::cout << "[Two Stack Queue] New Front: " << q.peek() << std::endl;
    std::cout << "[Two Stack Queue] Is empty? " << (q.empty() ? "true" : "false") << std::endl;

    return 0;
}
```



3. Practical Application & Problem Solving

• Linked List Queue: Ideal for dynamic memory environments where size varies unpredictably (e.g., CPU task scheduling, print queues)

• Two-Stack Queue: Useful in interview settings and system designs where only LIFO primitives (stacks) are available, or in algorithms like Breadth-First Search (BFS)

4. Analysis of Time and Space Complexity

Approach / Operation	Time Complexity	Justification
Linked List (Enqueue / Dequeue)	O(1)	Direct pointer updates at head and tail allow constant time insertion and deletion
Two Stacks (Enqueue)	O(1)	Elements are simply pushed onto s1
Two Stacks (Dequeue / Peek)	Amortized O(1)	Each element is pushed and popped between stacks at most twice across operations

• Space Complexity: O(N) for both implementations to store N elements